Tema 5

Uso de estructuras de control

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Uso de estructuras de control 5
3. Estructuras de selección 7
3.1 Operadores lógicos 9
4. Estructuras de repetición 13
4.1 Bucle while 13
4.2 Bucle For 15
5. Estructuras de salto 17
6. Resumen 20
Glosario 21

Objetivos de la unidad

- Controlar el flujo de ejecución: Las estructuras de control nos permiten dirigir el flujo de ejecución de un programa. Podemos decidir qué instrucciones se ejecutan y en qué orden, lo que nos brinda un mayor control sobre cómo se procesa y manipula la información.

- Tomar decisiones: Las estructuras de control condicional, como el **if** y el **else**, nos permiten tomar decisiones basadas en ciertas condiciones. Esto nos permite ejecutar diferentes bloques de código según el resultado de una evaluación lógica, como verificar si una condición es verdadera o falsa.

- Repetir acciones: Las estructuras de control de bucle, como el **for** y el **while**, nos permiten repetir acciones o bloques de código múltiples veces. Esto es útil cuando necesitamos realizar una tarea varias veces sin tener que repetir manualmente el código.

- Optimizar el código: Al utilizar estructuras de control adecuadas, podemos escribir código más limpio y legible. Esto hace que el código sea más fácil de entender, mantener y depurar. Además, nos permite evitar la repetición innecesaria de código, ya que podemos utilizar bucles para ejecutar tareas similares de manera eficiente.

- Crear lógica compleja: Al combinar diferentes estructuras de control, podemos crear lógica compleja en nuestros programas. Podemos encadenar condiciones y bucles para controlar situaciones más sofisticadas y tomar decisiones basadas en múltiples factores.

Tech Hub <FP> euroformac

1. Introducción

Los **algoritmos** que hemos visto hasta ahora consisten en simples secuencias de comandos. Sin embargo, hay tareas complejas que no se pueden resolver de esta forma tan sencilla y, en ocasiones, es necesario repetir la misma tarea un cierto número de veces o evaluar una expresión y realizar una acción.

De acuerdo con los resultados de la evaluación anterior. Para hacer frente a esta situación, existen estructuras de gestión con las siguientes características:

- Una estructura de control tiene un punto de entrada y un punto de salida.
- Las estructuras de control consisten en sentencias u otras estructuras de control.

Estas características permiten una creación muy flexible de todo tipo de algoritmos, incluso cuando solo hay tres tipos básicos de estructuras de control.

- Selección
- Repetición.
- Salto.

Esta unidad presenta las diversas estructuras de control, cómo expresarlas en notación algorítmica y las sentencias asociadas para usarlas en nuestros programas.

Las estructuras de control son elementos esenciales en la programación. Permiten controlar el flujo de ejecución de un programa, tomar decisiones y repetir acciones. (“ Guía de 【 Estructuras de Control de Flujo - ewebik.com”) Estas estructuras nos brindan la capacidad de crear programas más dinámicos y adaptativos.

2. Uso de estructuras de control

Los **programas definidos** hasta ahora se ejecutan **secuencialmente**, es decir, instrucción por instrucción. La ejecución comienza con la primera instrucción del programa y continúa hasta la última instrucción.

Se ejecuta sólo una vez. Este tipo de programación es ideal para programas simples. Sin embargo, resolver problemas comunes requiere la capacidad de controlar qué comandos se ejecutan y cuándo.

Una estructura de control o estructura de control define la secuencia o flujo de ejecución de comandos. Las estructuras de control se dividen en **tres categorías** según el flujo de ejecución:

- **Selección**
- **Iteración** o **repetición**
- **Salto**

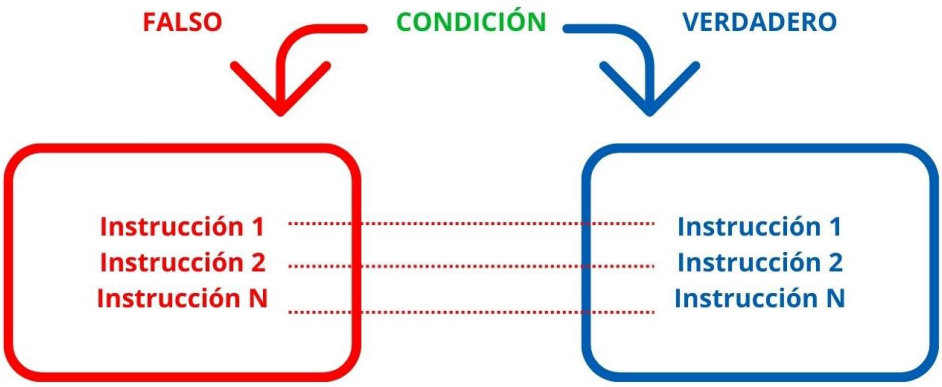
En Python, las estructuras de control están restringidas por bloques de código fijos con **sangría** proporcional (la misma sangría para cada estructura).

En Python, **la sangría es esencial y se utiliza para delimitar bloques de código**. A diferencia de otros lenguajes de programación que utilizan llaves o palabras clave para indicar el comienzo y el final de un bloque, en Python se utiliza la sangría.

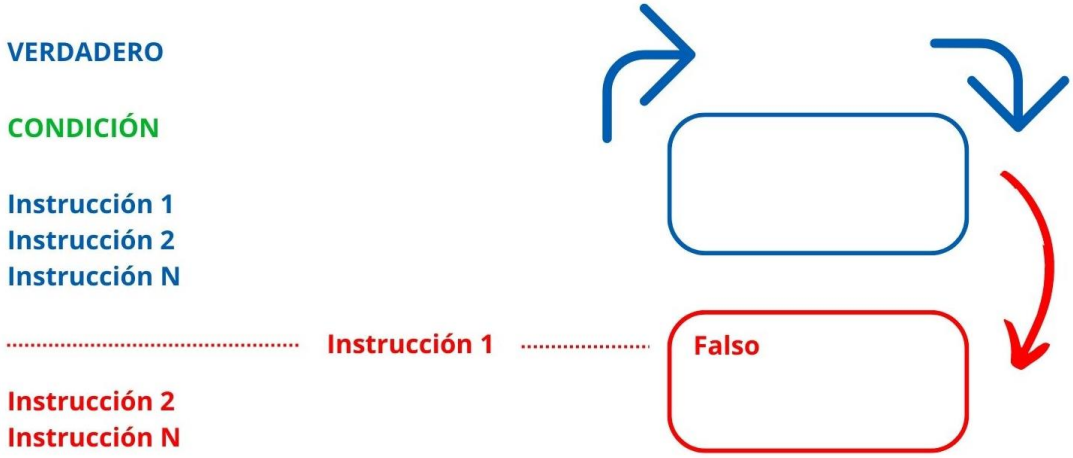
La sangría consiste en **indentar** el código mediante espacios o tabulaciones. Por convención, se recomienda utilizar cuatro espacios para cada nivel de sangría.

La estructura de control se define gráficamente de la siguiente manera:

Control de selección:



Control de repetición:



Control de salto:

3. Estructuras de selección

Estructuras de control **condicional** o **selección.** Cuando estamos programando puede ser útil detenernos y hacernos preguntas, ya que algunas partes de los algoritmos tienen múltiples opciones para continuar.

En Python, la estructura de selección `**if-then-else**` se utiliza para ejecutar un bloque de código si se cumple una condición y otro bloque de código en caso contrario.

Aquí tenemos un ejemplo básico:

```
x = 10
if x > 0:
    print("x es positivo")
else:
    print("x no es positivo")
```

En este ejemplo, se evalúa la condición `**x > 0**`. Si la condición es verdadera, se ejecuta el bloque de código indentado después del `**if**`, que en este caso imprime "x es positivo". Si la condición es falsa, se ejecuta el bloque de código indentado después del `**else**`, que imprime "x no es positivo".

Tenemos en cuenta que la estructura `**else**` es opcional. Puedes usar sólo `if` si sólo necesitamos tomar una decisión basada en una condición, sin una alternativa en caso de que la condición sea falsa.

También podemos encadenar múltiples condiciones utilizando `**elif**` (contracción de "**else if**") para evaluar diferentes casos.

Aquí tenemos un ejemplo:

```
x = 10
if x > 0:
    print("x es positivo")
elif x == 0:
    print("x es cero")
else:
    print("x es negativo")
```

En este caso, si la primera condición `x > 0` es falsa, se evalúa la siguiente condición `x == 0`. Si esa condición también es falsa, se ejecuta el bloque de código después del `else`. Es importante **tener en cuenta el orden de**

evaluación, ya que se ejecutará el primer bloque de código cuya condición sea verdadera.

Recordemos que la **indentación** adecuada es fundamental en Python para delimitar los bloques de código asociados a las estructuras de selección.

En Python, puedes utilizar la estructura **if-elif-else** para manejar selecciones múltiples. A continuación, mostramos cómo se utiliza:

- **If-elif-else simple:**

```
x = 10
if x > 0:
    print("x es positivo")
elif x == 0:
    print("x es cero")
else:
    print("x es negativo")
```

En este ejemplo, se evalúan las condiciones en orden. Si la primera condición x > 0 es verdadera, se ejecuta el bloque de código asociado y se omite el resto. Si la primera condición es falsa, se evalúa la siguiente condición x == 0. Si también es verdadera, se ejecuta su bloque de código y se omite el resto. Si ninguna de las condiciones anteriores es verdadera, se ejecuta el bloque de código asociado al else.

- **If-elif-else múltiple:**

```
x = 10
if x == 0:
    print("x es cero")
elif x == 1:
    print("x es uno")
elif x == 2:
    print("x es dos")
else:
    print("x es un número distinto de cero, uno o dos")
```

En este ejemplo, se evalúan múltiples condiciones utilizando **elif** para cada caso. Si alguna de las condiciones es verdadera, se ejecuta su bloque de código asociado y se omite el resto. Si ninguna de las condiciones anteriores es verdadera, se ejecuta el bloque de código asociado al else.

Recordemos que la estructura **else** es opcional y podemos omitirla si no necesitamos un bloque de código alternativo en caso de que ninguna de las condiciones sea verdadera.

3.1 Operadores lógicos

En Python, existen operadores lógicos de comparación que se utilizan para comparar valores y devuelven un resultado lógico (`True` o `False`) según se cumpla o no la condición. A continuación, se describen los **principales operadores lógicos de comparación**:

- **Operador `==` (igualdad):** Compara si dos valores son iguales. Devuelve `True` si los valores son iguales y `False` en caso contrario. Aquí tenemos un ejemplo:

```
x = 5
y = 10

if x == y:
    print("x es igual a y")
else:
    print("x no es igual a y")
```

En este caso, como `x` es diferente de `y`, se imprime "x no es igual a y".

- **Operador `!=` (desigualdad):** Compara si dos valores son diferentes. Devuelve `True` si los valores son diferentes y `False` en caso contrario. Aquí tenemos un ejemplo:

```
x = 5
y = 10

if x != y:
    print("x es diferente de y")
else:
    print("x es igual a y")
```

En este caso, como `x` es diferente de `y`, se imprime "x es diferente de y".

- **Operadores de comparación numérica:** Además de la igualdad y la desigualdad, existen operadores de comparación numérica para comparar si

un valor es mayor, menor, mayor o igual, o menor o igual a otro valor. Aquí tenemos ejemplos de cada operador:

```
x = 5
y = 10

if x > y:
    print("x es mayor que y")

if x < y:
    print("x es menor que y")

if x >= y:
    print("x es mayor o igual que y")

if x <= y:
    print("x es menor o igual que y")
```

En este caso, se imprimirá "x es menor que y" y "x es menor o igual que y", ya que `x` es menor que `y` pero no es mayor ni igual.

A continuación, se muestran en una tabla los operadores lógicos de comparación más comunes en Python. Podemos utilizarlos para realizar comparaciones entre valores y construir condiciones lógicas en tus programas. Recordamos que el resultado de una comparación es un valor booleano (`True` o `False`) que los podemos utilizar en estructuras de control como `if` o `while` para tomar decisiones basadas en las comparaciones.

Símbolo	Significado	Ejemplo	Resultado
==	Igual que	5 == 7	Falso
!=	Distinto que	rojo != verde	Verdadero
<	Menor que	8 < 12	Verdadero
>	Mayor que	7 > 12	Falso
<=	Menor o igual que	12 <= 12	Verdadero
>=	Mayor o igual que	4 >= 5	Falso

En Python, existen **tres operadores lógicos principales** que se utilizan para combinar y evaluar expresiones lógicas. Estos operadores son:

- **Operador `and` (y lógico):** Devuelve `True` si todas las expresiones que se evalúan son verdaderas. Si al menos una de las expresiones es falsa, el resultado es `False.

```
x = 5
y = 10

if x > 0 and y < 20:
    print("Ambas condiciones son verdaderas")
else:
    print("Al menos una de las condiciones es falsa")
```

En este caso, ambas condiciones `x > 0` y `y < 20` deben ser verdaderas para que se imprima "Ambas condiciones son verdaderas". Si alguna de las condiciones es falsa, se imprime "Al menos una de las condiciones es falsa".

- **Operador `or` (o lógico):** Devuelve `True` si al menos una de las expresiones que se evalúan es verdadera. Si todas las expresiones son falsas, el resultado es `False`.

```
x = 5
y = 10
if x > 0 or y > 20:
    print("Al menos una de las condiciones es verdadera")
else:
    print("Ambas condiciones son falsas")
```

En este caso, si al menos una de las condiciones `x > 0` o `y > 20` es verdadera, se imprime "Al menos una de las condiciones es verdadera". Sólo cuando ambas condiciones sean falsas se imprimirá "Ambas condiciones son falsas".

- **Operador `not` (negación lógica):** Devuelve el valor opuesto a la expresión que se evalúa. Si la expresión es `True`, el resultado es `False`, y si la expresión es `False`, el resultado es `True`.

```
x = 5
if not x > 10:
    print("La expresión es falsa")
```

```
else:
    print("La expresión es verdadera")
```

En este caso, la expresión `not x > 10` evalúa si `x` es mayor que 10 y luego niega el resultado. Como `x` es igual a 5 y no es mayor que 10, la expresión es falsa y se imprime "La expresión es falsa".

Estos operadores lógicos son útiles para combinar expresiones lógicas y construir condiciones más complejas en tus programas. Podemos utilizar paréntesis para agrupar expresiones y establecer el orden de evaluación según sea necesario.

A continuación, se muestra una tabla que recoge todos los casos posibles:

Símbolo	Ejemplo	Resultado
and (y)	5 == 7 and 7 < 12	0 y 1 Falso
	9 < 12 and 12 > 7	1 y 1 Verdadero
	9 < 12 and 12 > 15	1 y 0 Verdadero
or (o)	12 == 12 or 15 < 7	1 y 0 Falso
	7 > 5 or 9 < 12	1 y 1 Verdadero
xor*	(4 == 4) ^ (9 > 3)	1 y 1 Verdadero
(o excluyente)	(4 == 4) ^ (9 < 3)	1 y 0 Falso

4. Estructuras de repetición

El mismo bloque de código puede ejecutarse N veces siempre que se cumpla una condición gracias a estructuras de control iterativas (también conocidas como cíclicas o bucles).

Python dispone dos estructuras de control iterativas:

- El bucle **while**
- El bucle **for**

4.1 Bucle while

Llamado así porque while nos permite ejecutar una pieza de código repetidamente. El código se ejecuta hasta que se cumplen ciertas condiciones. Cuando ya no es verdadero, el bucle sale y continúa la ejecución normal. La ejecución completa de un bloque de código se denomina **iteración**.

Mostramos a continuación algunos ejemplos adicionales de cómo se puede utilizar el bucle "**while**" en Python:

- **Sumar números del 1 al 10:**

```
suma = 0
num = 1
while num <= 10:
    suma += num
    num += 1
print("La suma es:", suma)
```

Este código utiliza un bucle "while" para sumar los números del 1 al 10. En cada iteración, se agrega el valor actual de "num" a la variable "suma", y luego "num" se incrementa en 1. Al final, se imprime el resultado de la suma.

- **Contar cuántas veces se puede dividir un número por 2 hasta que sea menor que 1:**

```
num = 100
contador = 0
while num >= 1:
    num /= 2
    contador += 1
print("El número se pudo dividir", contador, "veces por 2
hasta ser menor que 1.")
```

En este ejemplo, se utiliza un bucle "while" para contar cuántas veces se puede dividir un número (inicialmente 100) por 2 hasta que sea menor que 1. En cada iteración, el número se divide por 2 y se incrementa el contador. El bucle continúa hasta que el número sea menor que 1, momento en el que se imprime el contador.

- **Validar la entrada de un número positivo:**

```
numero = -1
while numero < 0:
    numero = int(input("Ingrese un número positivo: "))
print("Número válido ingresado:", numero)
```

En este ejemplo, se utiliza un bucle "while" para solicitar al usuario que ingrese un número positivo. Si el número ingresado es menor que 0, el bucle se repite y solicita nuevamente la entrada. Una vez que se ingresa un número válido (mayor o igual a 0), el bucle se detiene y se imprime el número válido.

Estos son solo algunos ejemplos de cómo se puede utilizar el bucle "while" en Python. La versatilidad de este bucle permite adaptarlo a diferentes situaciones y condiciones específicas.

Sabías que

Sucesión de **Fibonacci** en Python. En el ámbito de las matemáticas, la sucesión de Fibonacci se refiere a una serie infinita de números enteros positivos, en la cual cada término se genera mediante la suma de los dos números precedentes.

```
a, b = 0, 1
while b < 25:
    print(b)
    a, b = b, a + b

#1, 1, 2, 3, 5, 8, 13, 21
```

4.2 Bucle For

Un tipo de bucle **for**, similar a **while**, pero con algunas diferencias. **El principal problema es que el número de iteraciones en un for está predeterminado, mientras que en un while no lo está. La principal diferencia con while está en la condición.** Mientras que en el bucle while, se evalúa una condición en cada iteración para determinar si se reinicia o no el código, en el bucle for no habrá tal condición, sino una condición de iteración que determina cuántas veces se ejecutará el código.

En el siguiente ejemplo podemos ver que el ciclo for se ejecuta 5 veces. donde i "automáticamente" incrementa el valor en 1 cada iteración.

```
for i in range(0, 5):
    print(i)

# Salida:
# 0
# 1
# 2
# 3
# 4
```

Otra forma de iterar con el bucle for, puede emular a while:

Por cada año en el rango 2011 a 2023, imprimir la frase "Informes de <año>".

```
for year in range(2011, 2023):
    print("Informes de", year)
```

Otros ejemplos de bucle for:

- **Imprimir los elementos de una lista:**

```
frutas = ["manzana", "banana", "cereza"]
for fruta in frutas:
    print(fruta)
```

En este ejemplo, el bucle "for" se utiliza para iterar a través de los elementos de la lista "frutas". En cada iteración, el valor actual se asigna a la variable "fruta" y se imprime. El bucle se repite hasta que se hayan recorrido todos los elementos de la lista.

- **Calcular la suma de los números del 1 al 10:**

```
suma = 0
for num in range(1, 11):
    suma += num
print("La suma es:", suma)
```

En este caso, se utiliza la función "range()" para generar una secuencia de números del 1 al 10. El bucle "for" itera a través de estos números y, en cada iteración, se agrega el número actual a la variable "suma". Al final, se imprime el resultado de la suma.

- **Imprimir los caracteres de una cadena de texto:**

```
texto = "Hola, mundo!"
for caracter in texto:
    print(caracter)
```

Aquí, el bucle "for" se utiliza para recorrer cada carácter de la cadena de texto "texto". En cada iteración, el valor actual se asigna a la variable "caracter" y se imprime. Esto permite mostrar cada letra o símbolo del texto por separado.

Estos ejemplos ilustran algunas de las formas en que puedes utilizar el bucle "for" en Python. Es una herramienta muy útil para iterar a través de listas, secuencias numéricas, cadenas de texto y otros tipos de datos iterables.

Visualización de los elementos de una matriz con bucles for anidados

```
matriz = [[-5,2,0], [9,5,6], [1,7,15]]
for fila in matriz:
    for elemento in fila:
        print(elemento)
```

```
-5
2
0
9
5
6
1
7
15
```

5. Estructuras de salto

Definición

Las instrucciones de salto nos permiten saltar el orden de ejecución del programa, es decir, nos permiten transferir el control del programa y cambiar repentinamente el orden de ejecución.

Existen **cuatro tipos de instrucciones de salto**:

- Break (romper, salir, terminar, …)
- Continue
- Pass

En Python, **existen tres estructuras de salto principales** que puedes utilizar para controlar el flujo de ejecución de un programa. Estas **estructuras de salto** son:

- **Declaración "break":**

La declaración "break" se utiliza para salir de un bucle de forma prematura. Cuando se ejecuta la declaración "break", el programa sale inmediatamente del bucle y continúa con la siguiente línea de código después del bucle. Aquí tienes un ejemplo que muestra cómo se utiliza:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

En este ejemplo, el bucle "for" itera a través de los números del 1 al 9. Si el valor de "i" es igual a 5, se ejecuta la declaración "break" y el bucle se interrumpe. Por lo tanto, solo se imprimirán los números del 1 al 4.

- **Declaración "continue":**

La declaración "continue" se utiliza para pasar a la siguiente iteración de un bucle de forma prematura, sin ejecutar el resto del bloque de código en esa iteración en particular. Cuando se ejecuta la declaración "continue", el programa salta a la siguiente iteración del bucle.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

En este ejemplo, el bucle "for" itera a través de los números del 1 al 5. Si el valor de "i" es igual a 3, se ejecuta la declaración "continue" y se pasa a la siguiente iteración sin imprimir ese número. Por lo tanto, se imprimirán los números 1, 2, 4 y 5.

- **Declaración "pass":**

En Python, la instrucción "pass" se utiliza como marcador de posición cuando necesitas una declaración sintácticamente válida pero no deseas ejecutar ninguna instrucción en ese momento. La instrucción "pass" no hace nada y se utiliza principalmente como un marcador temporal para evitar errores de sintaxis cuando se está desarrollando código.

Mostramos algunos casos comunes donde puedes usar la instrucción "pass":

- **Definir una función o una clase vacía:**

```
def mi_funcion():
  pass

class MiClase:
  pass
```

En este ejemplo, se definen una función llamada "mi_funcion()" y una clase llamada "MiClase". Si aún no has implementado el cuerpo de la función o la clase, puedes usar la instrucción "pass" como marcador de posición hasta que agregues el código real.

- **En una estructura condicional donde no se requiere ninguna acción para una rama específica:**

```
if condicion:
  # Código para la rama verdadera
else:
  pass
```

Aquí, si la condición es falsa y no necesitas realizar ninguna acción en la rama "else", puedes utilizar la instrucción "pass" como marcador de posición.

- **En un bucle donde no se requiere ninguna acción en una iteración específica:**

```
for elemento in lista:

if condicion:
    # Código para la rama verdadera
else:
    pass
```

En este caso, si se cumple una condición en el bucle y no deseas realizar ninguna acción en esa iteración particular, puedes usar la instrucción "pass" como marcador de posición.

En resumen, la instrucción "pass" se utiliza como marcador de posición para evitar errores de sintaxis cuando se requiere una declaración válida pero no se necesita ninguna acción en ese momento. Es útil durante el desarrollo del código cuando se trabaja en partes del programa que aún no se han implementado por completo.

6. Resumen

Los conceptos más relevantes de la unidad son:

- Estructuras de selección: La estructura **"if"** se emplea para ejecutar un bloque de código si se cumple una condición determinada.
- La estructura **"if-else"** se emplea para ejecutar un bloque de código si se cumple una condición, y otro bloque de código si la condición no se cumple.
- La estructura **"if-elif-else"** se emplea para evaluar múltiples condiciones y ejecutar diferentes bloques de código según la primera condición que se cumpla.
- Estructuras de repetición: El bucle **"while"** se utiliza para repetir un bloque de código mientras se cumpla una condición específica.
- El bucle **"for"** se utiliza para iterar sobre una secuencia de elementos, como una lista o una cadena de texto, y ejecutar un bloque de código para cada elemento.
- Estructuras de salto: La declaración **"break"** se utiliza para salir de un bucle de forma prematura.
- La declaración **"continue"** se utiliza para pasar a la siguiente iteración de un bucle sin ejecutar el resto del bloque de código en esa iteración.
- La declaración **"return"** se utiliza para salir de una función y opcionalmente devolver un valor.

Estas estructuras de control son fundamentales en Python y te permiten tomar decisiones, repetir bloques de código y controlar el flujo de ejecución de un programa. Con ellas, puedes construir algoritmos más complejos y resolver una amplia variedad de problemas de programación.

Glosario

- **Bucle:** Un bucle es una serie de instrucciones que ejecuta repetidamente la misma línea de código hasta que la condición que el bucle debe manejar ya no se cumple.
- **Indentación:** Consiste en añadir espacios o tabulaciones al comienzo de una línea de código o bloque de instrucciones para indicar su relación con una estructura superior.
- **Instrucción:** En programación, una instrucción es una unidad básica de código que realiza una acción específica. Una instrucción es una orden o comando que le indica al programa qué hacer. Las instrucciones pueden ser simples o complejas, dependiendo del lenguaje de programación y de la tarea que se esté realizando.
- **Iteración:** en el mundo de la programación, la iteración se refiere a la repetición de un conjunto de instrucciones, características o actividades una o más veces hasta que se cumple una condición o ya no se cumple.
- **Matriz:** En programación, una matriz es una organización de datos que guarda un conjunto secuencial de elementos homogéneos. Además, es reconocida bajo el nombre de. array. Una matriz se representa como una tabla rectangular compuesta por filas y columnas
- **Operador:** En programación, un operador es un símbolo o palabra reservada que indica una operación específica a realizar en uno o más operandos. Los operadores se utilizan para realizar cálculos, comparaciones y manipulaciones de datos en un programa. Los operadores pueden ser aritméticos, de asignación, de comparación, lógicos, de concatenación de cadenas, entre otros.